Derived Datatypes

Victor Eijkhout
Jack Gaither

2026 PCSE

Overview

In this section you will learn about derived data types.

Commands learned:

- *MPI_Type_contiguous*/`vector/indexed/struct` *MPI_Type_create_subarray*
- *MPI_Pack* / *MPI_Unpack*
- F90 types

Motivation: datatypes in MPI

All examples so far:

- contiguous buffer
- elements of single type

We need data structures with gaps, or heterogeneous types.

- Send real or imaginary parts out of complex array.
- Gather/scatter cyclicly.
- Send `struct` or *Type* data.

MPI allows for recursive construction of data types.

Datatype topics

- Elementary types: built-in.
- Derived types: user-defined.
- Packed data: not really a datatype.

Elementary datatypes

C/C++	Fortran
MPI_CHAR	*MPI_CHARACTER*
MPI_UNSIGNED_CHAR	
MPI_SIGNED_CHAR	
	MPI_LOGICAL
MPI_SHORT	
MPI_UNSIGNED_SHORT	
MPI_INT	*MPI_INTEGER*
MPI_UNSIGNED	
MPI_LONG	
MPI_UNSIGNED_LONG	
MPI_FLOAT	*MPI_REAL*
MPI_DOUBLE	*MPI_DOUBLE_PRECISION*
MPI_LONG_DOUBLE	
	MPI_COMPLEX
	MPI_DOUBLE_COMPLEX

How to use derived types

Create, commit, use, free:

```
1 MPI_Datatype newtype;
2 MPI_Type_xxx( ... oldtype ... &newtype);
3 MPI_Type_commit ( &newtype );
4
5   // code using the new type
6
7 MPI_Type_free ( &newtype );
```

```
1 Type(MPI_Datatype) :: newtype ! F2008
2 Integer            :: newtype ! F90
```

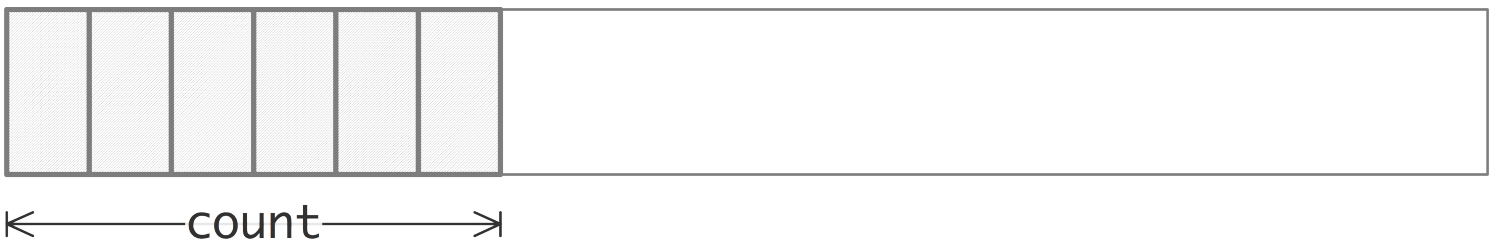
The *oldtype* can be elementary or derived.
Recursively constructed types.

MPL layouts

MPL derived types are derived classes from `mpl::layout`.
The commit call is done in the constructor.
The free call is done in the destructor.

Contiguous type

```
1 int MPI_Type_contiguous(
2   int count, MPI_Datatype old_type, MPI_Datatype *new_type_p)
```



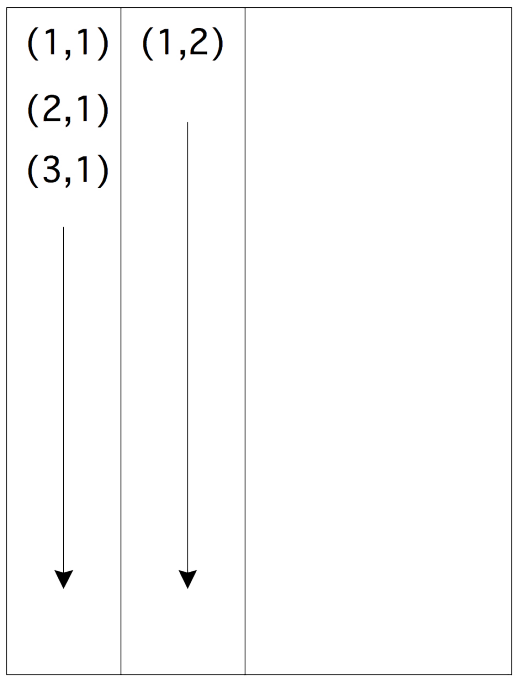
This one is indistinguishable from sending *count* instances of the *old_type*.

Example: non-contiguous data

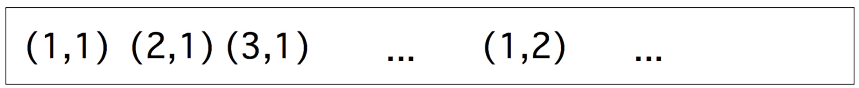
Matrix in column storage:

- Columns are contiguous
- Rows are not contiguous

Logical:

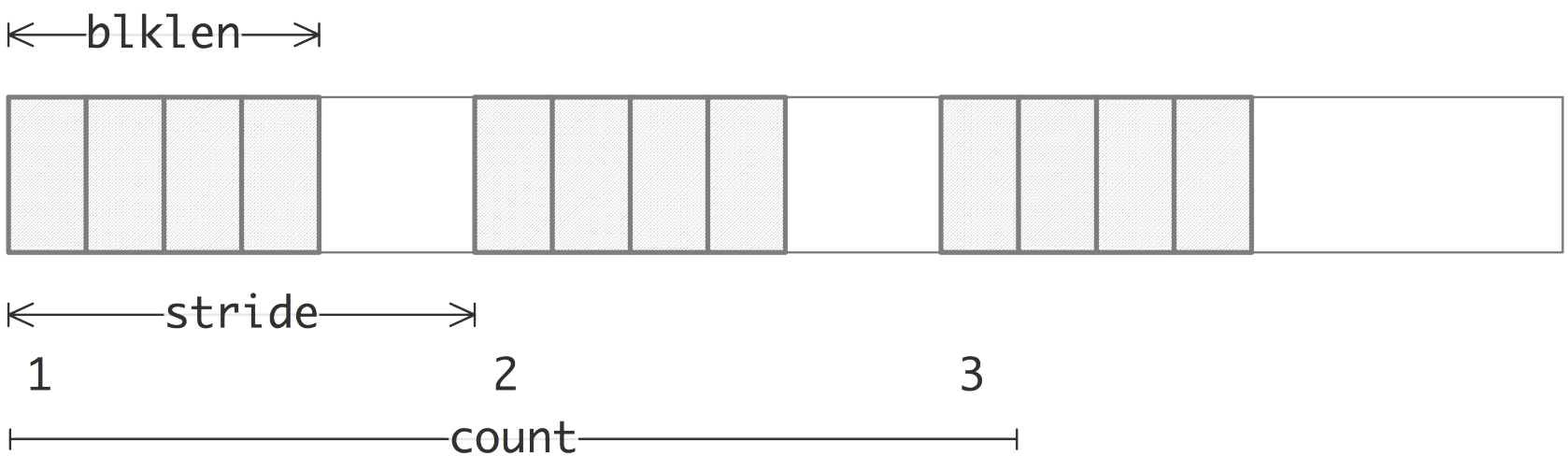


Physical:



Vector type

```
1 int MPI_Type_vector(
2   int count, int blocklength, int stride,
3   MPI_Datatype old_type, MPI_Datatype *newtype_p
4 );
```



Used to pick a regular subset of elements from an array.

MPL strided vector layout

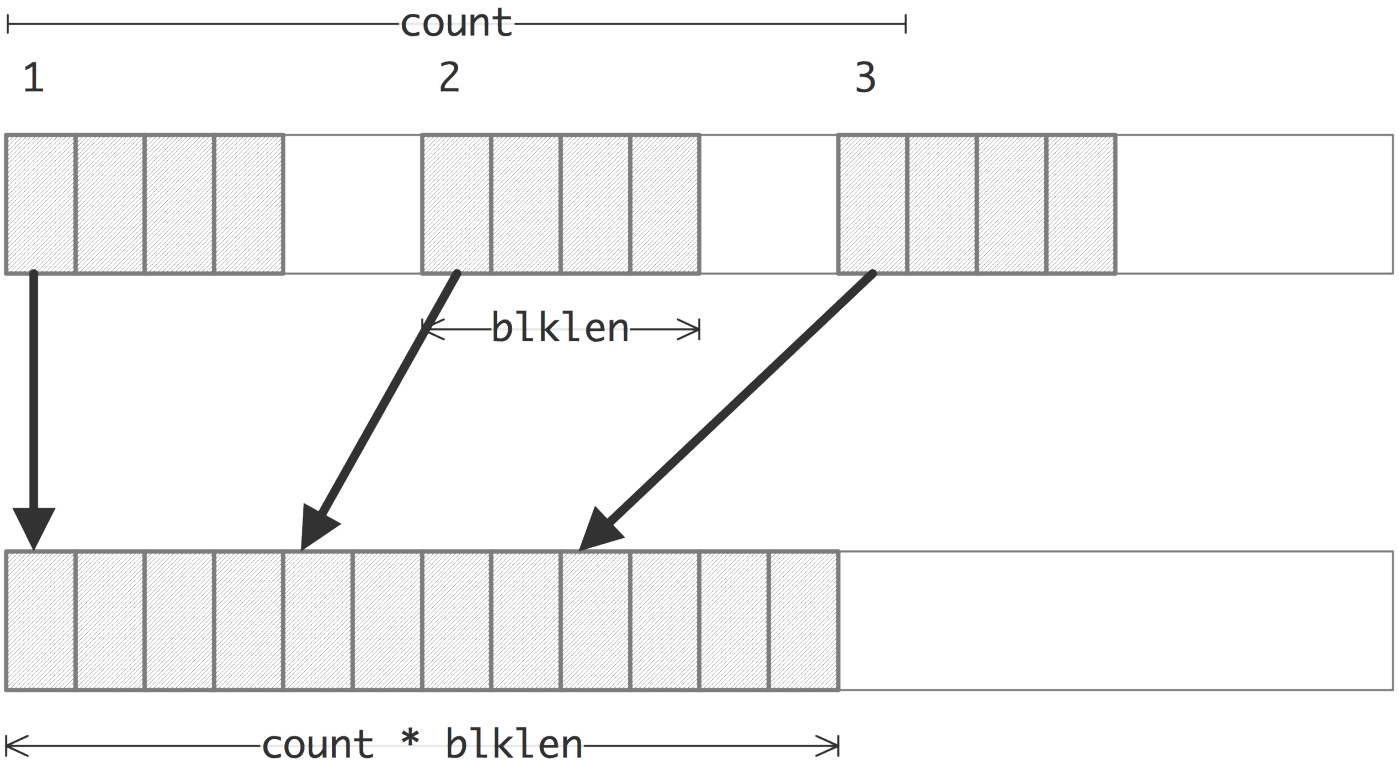
```
template<typename T>
class strided_vector_layout : public mpl::layout<T>

strided_vector_layout(int count, int blocklength, int stride);
strided_vector_layout(int count, int blocklength, int stride, const layout<T> &l);
```

```
// vector.cxx
vector<double>
  source(stride*count);
if (procno==sender) {
  mpl::strided_vector_layout<double>
    newvectortype(count,1,stride);
  comm_world.send
    (source.data(),newvectortype,the_other);
}
```

Different send and receive types

Send and receive type can differ. Example:
sender type: vector
receiver type: contiguous or elementary

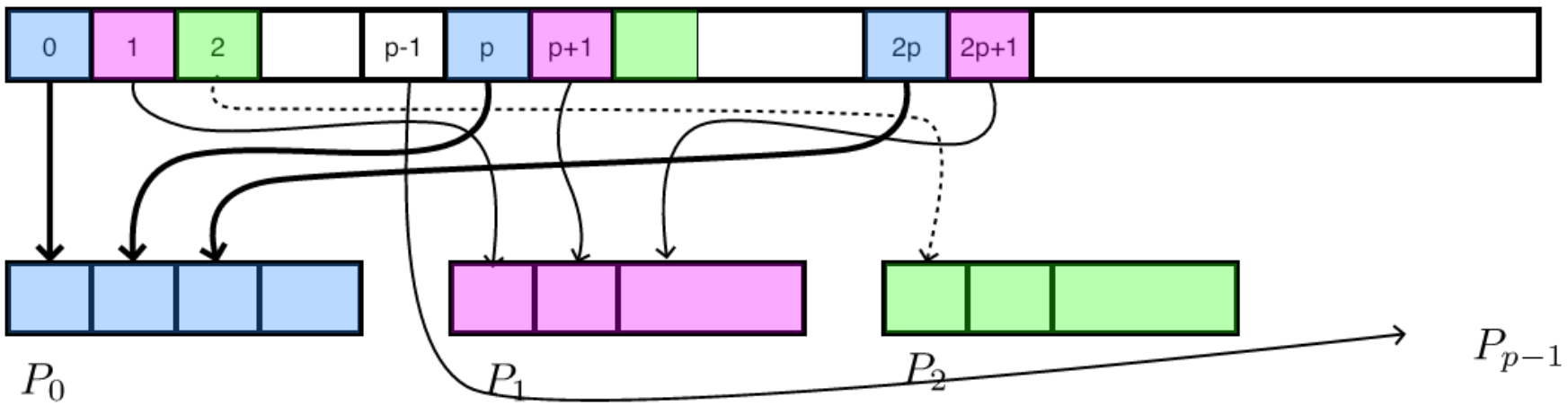


Receiver has no knowledge of the stride of the sender.

Send vs recv type

```
1  // vector.c
2  source = (double*) malloc(stride*count*sizeof(double));
3  target = (double*) malloc(count*sizeof(double));
4  MPI_Datatype newvectortype;
5  if (procno==sender) {
6    MPI_Type_vector(count,1,stride,MPI_DOUBLE,&newvectortype);
7    MPI_Type_commit(&newvectortype);
8    MPI_Send(source,1,newvectortype,the_other,0,comm);
9    MPI_Type_free(&newvectortype);
10 } else if (procno==receiver) {
11   MPI_Status recv_status;
12   int recv_count;
13   MPI_Recv(target,count,MPI_DOUBLE,the_other,0,comm,
14     &recv_status);
15   MPI_Get_count(&recv_status,MPI_DOUBLE,&recv_count);
16   ASSERT(recv_count==count);
17 }
```

Illustration of the next exercise



Sending strided data from process zero to all others

Exercise 1 `stridesend`

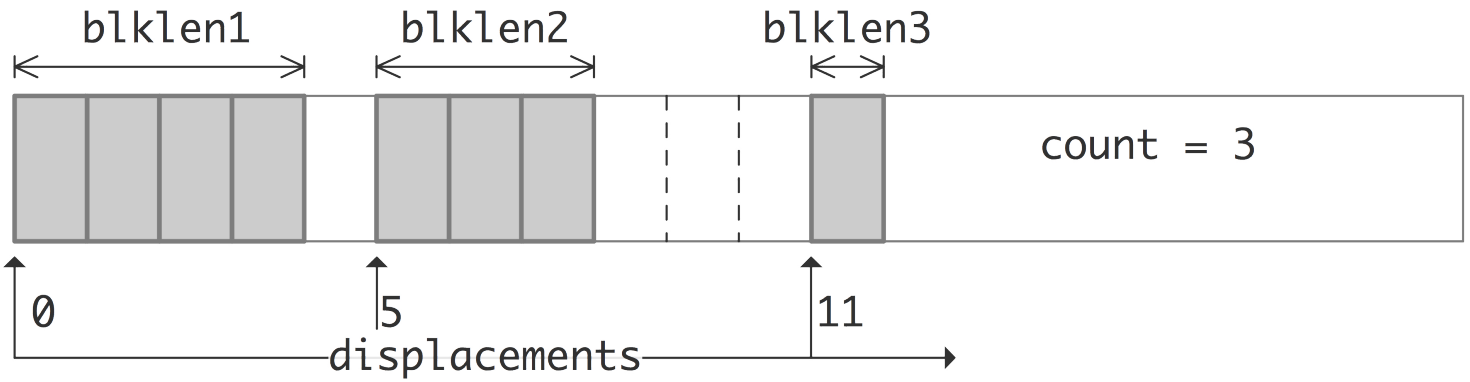
Let processor 0 have an array x of length $10P$, where P is the number of processors. Elements $0, P, 2P, \ldots, 9P$ should go to processor zero, $1, P+1, 2P+1, \ldots$ to processor 1, et cetera.

- Code this as a sequence of send/recv calls, using a vector datatype for the send, and a contiguous buffer for the receive.
- For simplicity, skip the send to/from zero. What is the most elegant solution if you want to include that case?
- For testing, define the array as $x[i] = i$.

Exercise (optional) 2

Allocate a matrix on processor zero, using Fortran column-major storage. Using P sendrecv calls, distribute the rows of this matrix among the processors.

Indexed type




```
1 int MPI_Type_indexed(
2   int count, int blocklens[], int displacements[],
3   MPI_Datatype old_type, MPI_Datatype *newtype);
```

Hindexed type

Similar to indexed but using byte offsets:
explicit memory address.

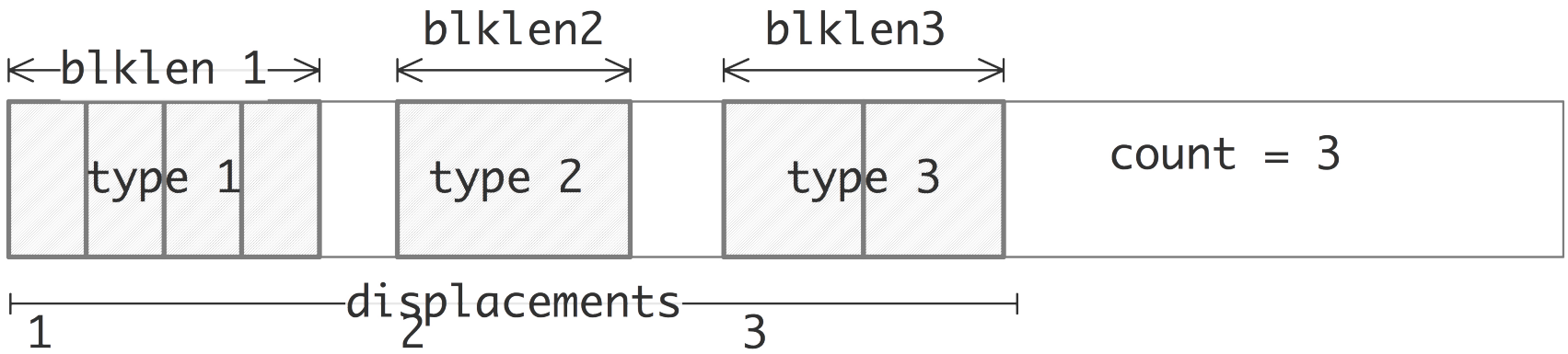
Example usage scenario: send linked list.
Use *MPI_Get_address* (figure 1)

Figure 1 MPI_Get_address

Name	Param name	Explanation	C type	F type
MPI_Get_address (				
	location	location in caller memory	const void*	TYPE(*), DIMENSION(.
	address	address of location	MPI_Aint*	INTEGER (KIND=MPI_A
	)			

Heterogeneous: Structure type

```
1 int MPI_Type_create_struct(
2   int count, int blocklengths[], MPI_Aint displacements[],
3   MPI_Datatype types[], MPI_Datatype *newtype);
```



This gets very tedious. . .

Packed data

Packing into buffer

- Construct a buffer with *MPI_Pack*
- Send with *MPI_Send* and such
- Receive
- Unpack buffer elements with *MPI_Unpack*

MPI_Pack

Name	Param name	Explanation	C type	F type
MPI_Pack (				
MPI_Pack_c (				
	inbuf	input buffer start	const void*	TYPE(*), DIMENSION(..)
	incount	number of input data items	int MPI_Count	INTEGER
	datatype	datatype of each input data item	MPI_Datatype	TYPE(MPI_Datatype)
	outbuf	output buffer start	void*	TYPE(*), DIMENSION(..)
	outsize	output buffer size, in bytes	int MPI_Count	INTEGER
	position	current position in buffer, in bytes	int* MPI_Count*	INTEGER
	comm	communicator for packed message	MPI_Comm	TYPE(MPI_Comm)
	)			

MPL: MPI_Pack

1 *Not available in MPL* 0.3

MPI_Unpack

Name	Param name	Explanation	C type	F type
MPI_Unpack (				
MPI_Unpack_c (				
	inbuf	input buffer start	const void*	TYPE(*), DIMENSION(..)
	insize	size of input buffer, in bytes	int MPI_Count	INTEGER
	position	current position in bytes	int* MPI_Count*	INTEGER
	outbuf	output buffer start	void*	TYPE(*), DIMENSION(..)
	outcount	number of items to be unpacked	int MPI_Count	INTEGER
	datatype	datatype of each output data item	MPI_Datatype	TYPE(MPI_Datatype)
	comm	communicator for packed message	MPI_Comm	TYPE(MPI_Comm)
	)			

MPL: MPI_Unpack

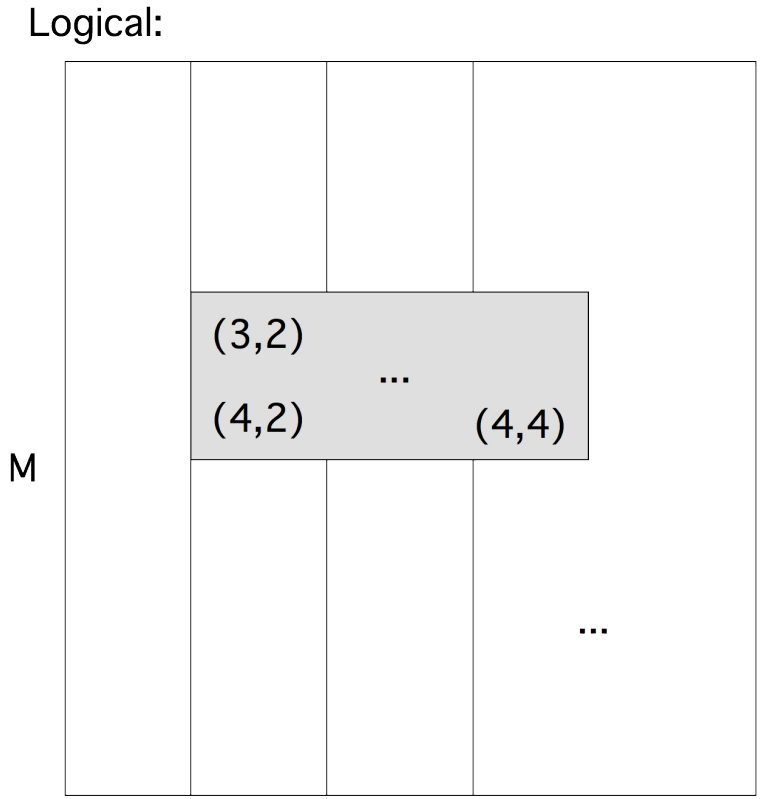
```
1 Not available in MPL 0.3
```

Example

```
1  if (procno==sender) {
2    position = 0;
3    MPI_Pack(&nsends,1,MPI_INT,
4             buffer,buflen,&position,comm);
5    for (int i=0; i<nsends; i++) {
6      double value = rand()/(double)RAND_MAX;
7      printf("[%d] pack %e\n",procno,value);
8      MPI_Pack(&value,1,MPI_DOUBLE,
9               buffer,buflen,&position,comm);
10   }
11   MPI_Pack(&nsends,1,MPI_INT,
12            buffer,buflen,&position,comm);
13   MPI_Send(buffer,position,MPI_PACKED,other,0,comm);
14 } else if (procno==receiver) {
15   int irecv_value;
16   double xrecv_value;
17   MPI_Recv(buffer,buflen,MPI_PACKED,other,0,
18            comm,MPI_STATUS_IGNORE);
19   position = 0;
20   MPI_Unpack(buffer,buflen,&position,
21              &nsends,1,MPI_INT,comm);
22   for (int i=0; i<nsends; i++) {
23     MPI_Unpack(buffer,buflen,
24                &position,&xrecv_value,1,MPI_DOUBLE,comm);
25     printf("[%d] unpack %e\n",procno,xrecv_value);
26   }
27   MPI_Unpack(buffer,buflen,&position,
28              &irecv_value,1,MPI_INT,comm);
29   ASSERT(irecv_value==nsends);
30 }
```

Subarray type

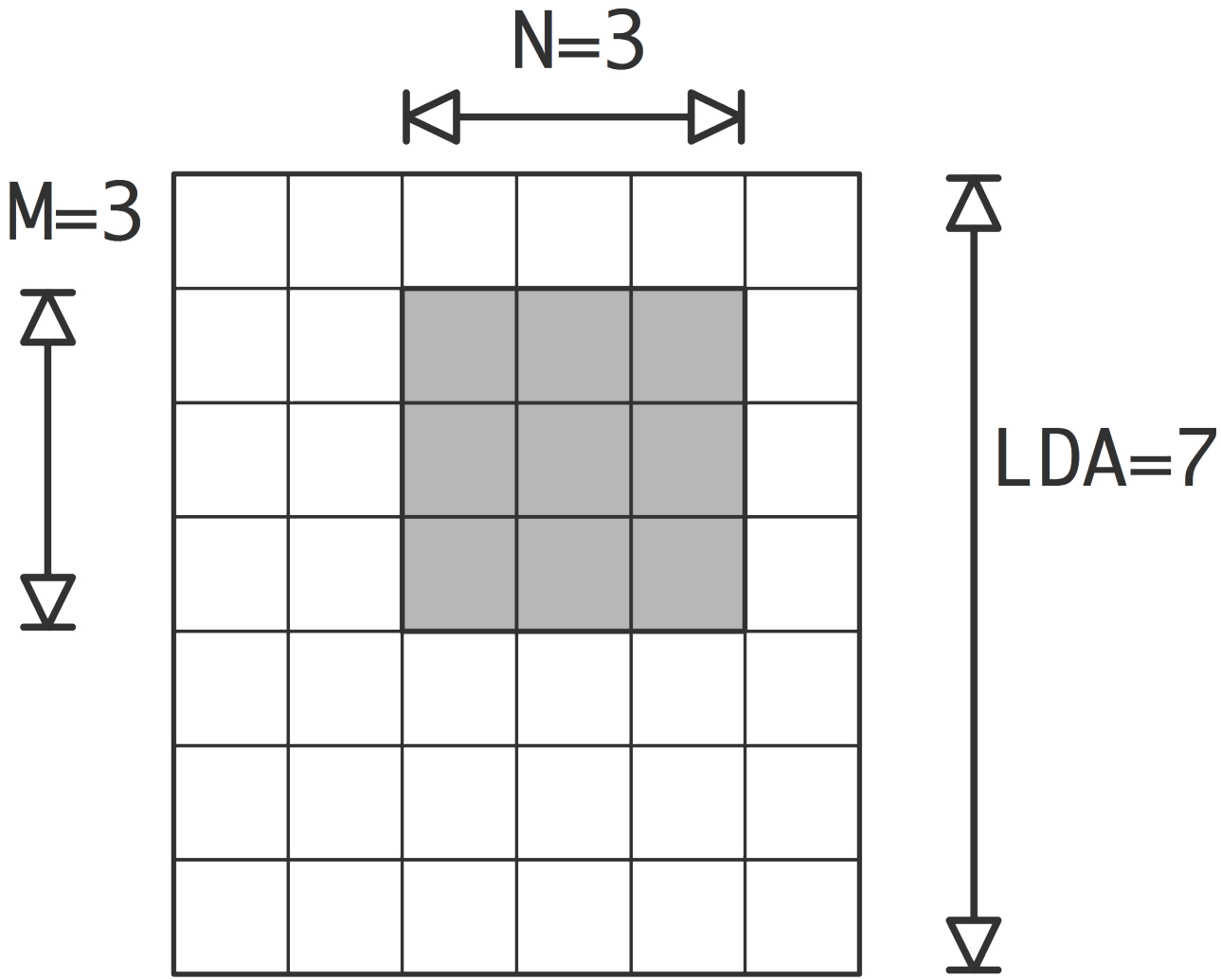
Submatrix storage



- Location of first element
- Stride, blocksize

BLAS/Lapack storage

Three parameter description:



How about as a 'block within a block'?

Subarray type

- Vector type is convenient for 2D subarrays,
- it gets tedious in higher dimensions.
- Better solution: *MPI_Type_create_subarray*

```
1 MPI_Type_create_subarray(
2     ndims, array_of_sizes, array_of_subsizes,
3     array_of_starts, order, oldtype, newtype)
```

Subtle: data does not start at the buffer start

Exercise 3 `cubegather`

Assume that your number of processors is $P = Q^3$, and that each process has an array of identical size. Use *`MPI_Type_create_subarray`* to gather all data onto a root process. Use a sequence of send and receive calls; *`MPI_Gather`* does not work here.

If you haven't started `idev` with the right number of processes, use

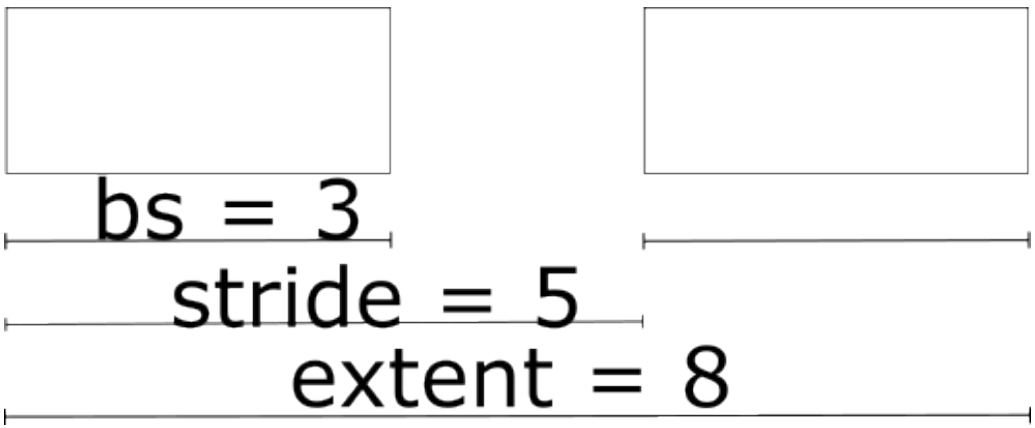
```
ibrun -np 27 cubegather
```

Normally you use `ibrun` without process count argument.

Extent and resizing

Extent

Extent: 'size' of a type,
especially useful for derived types.



Naive code

Send multiple derived types from

```
0 1 2 3 4 5 6 7 8 9 10
```

```
// vectorpadsend.c
int count = 3, stride = 2, blocklength = 1, ntypes = 2;
  for (int i=0; i<max_elements; i++) sendbuffer[i] = i;
  MPI_Type_vector(count,blocklength,stride,MPI_INT,&stridetype);
  MPI_Type_commit(&stridetype);
  MPI_Send( sendbuffer,ntypes,stridetype, receiver,0, comm );
```

Receive as single block:

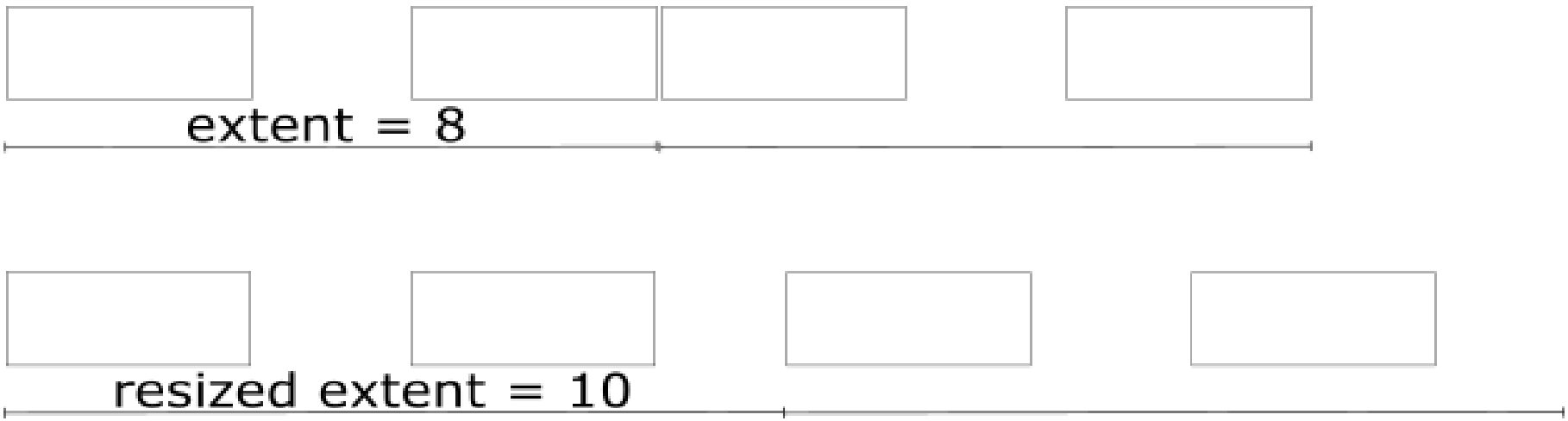
```
MPI_Recv( recvbuffer,max_elements,MPI_INT, sender,0, comm,&status );
int count; MPI_Get_count(&status,MPI_INT,&count);
printf("Receive %d elements:",count);
for (int i=0; i<count; i++) printf(" %d",recvbuffer[i]);
printf("\n");
```

giving an output of:

```
Receive 6 elements: 0 2 4 5 7 9
```

Extent resizing: enlarging

Multiple derived types may not be what you intended
extent resizing makes it artificually larger:



Extent computation

Use *MPI_Type_get_extent* to query extent

note: parameters are measured in bytes.

```
1 MPI_Aint lb,asize;
2 MPI_Type_vector(count,bs,stride,MPI_DOUBLE,&newtype);
3 MPI_Type_commit(&newtype);
4 MPI_Type_get_extent(newtype,&lb,&asize);
5 ASSERT( lb==0 );
6 ASSERT( asize==((count-1)*stride+bs)*sizeof(double) );
7 MPI_Type_free(&newtype);
```

Resizing code

Extend the vector type with padding:

```
1 MPI_Type_get_extent(stridetype,&l,&e);
2 printf("Stride type l=%ld e=%ld\n",l,e);
3 e += ( stride-blocklength) * sizeof(int);
4 MPI_Type_create_resized(stridetype,l,e,&paddedtype);
5 MPI_Type_get_extent(paddedtype,&l,&e);
6 printf("Padded type l=%ld e=%ld\n",l,e);
7 MPI_Type_commit(&paddedtype);
8 MPI_Send( sendbuffer,ntypes,paddedtype, receiver,0, comm );
```

giving:

```
Strided type l=0 e=20
Padded  type l=0 e=24
Receive 6 elements: 0 2 4 6 8 10
```

MPI_Type_create_resized

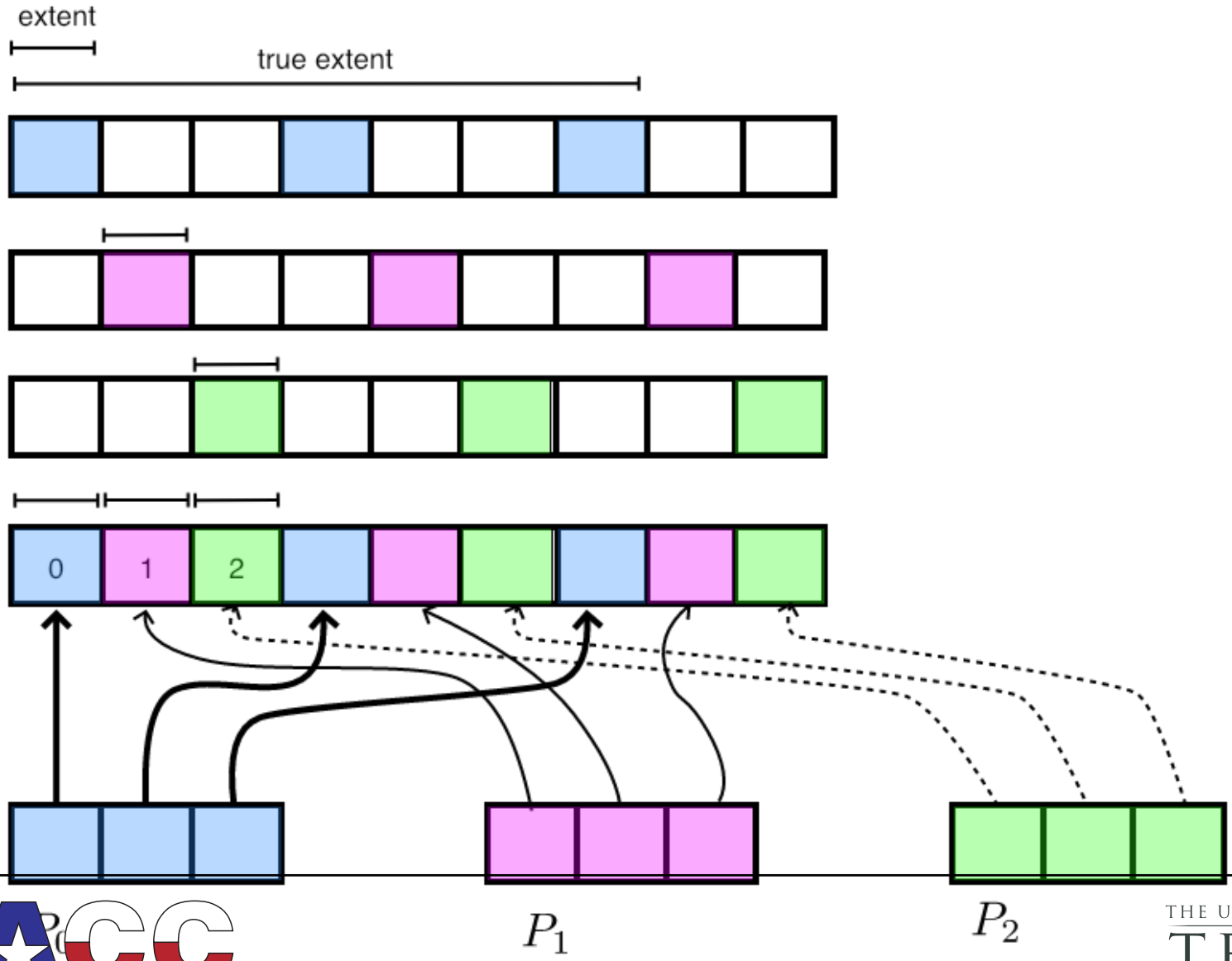
Name	Param name	Explanation	C type	F type
MPI_Type_create_resized (				
MPI_Type_create_resized_c (				
	oldtype	input datatype	MPI_Datatype	TYPE(MPI_Datatype)
	lb	new lower bound of datatype	[MPI_Aint MPI_Count	INTEGER (KIND=MPI_ADDRESS_KIND)
	extent	new extent of datatype	[MPI_Aint MPI_Count	INTEGER (KIND=MPI_ADDRESS_KIND)
	newtype	output datatype	MPI_Datatype*	TYPE(MPI_Datatype)
	)			

MPL: MPI_Type_create_resized

```
1 template<typename T>
2 class layout {
3
4 void resize(ssize_t lb, ssize_t extent);
5 void byte_resize(ssize_t lb, ssize_t extent);
```

Extent resizing: shrinking

Elements are placed at distance equal to extent:



Exercise 4 `stridescatter`

Change the `stridesend` code to use a scatter call, rather than a sequence of sends.

Extent of subarray type

The 'subarray' type:
data does not start at the start of the type.

MPI_Type_get_true_extent (figure 2)

Figure 2 MPI_Type_get_true_extent

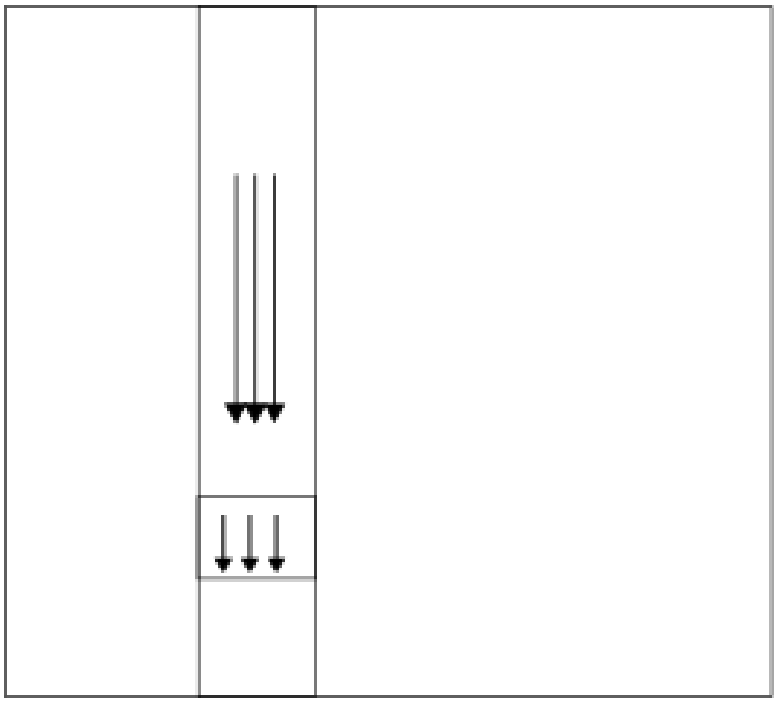
Name	Param name	Explanation	C type	F type
MPI_Type_get_true_extent (				
MPI_Type_get_true_extent_c (				
	datatype	datatype to get information on	MPI_Datatype	TYPE (MPI_Dataty
	true_lb	true lower bound of datatype	MPI_Aint* / MPI_Count*	INTEGER (KIND=MPI_A
	true_extent	true extent of datatype	MPI_Aint* / MPI_Count*	INTEGER (KIND=MPI_A
	)			

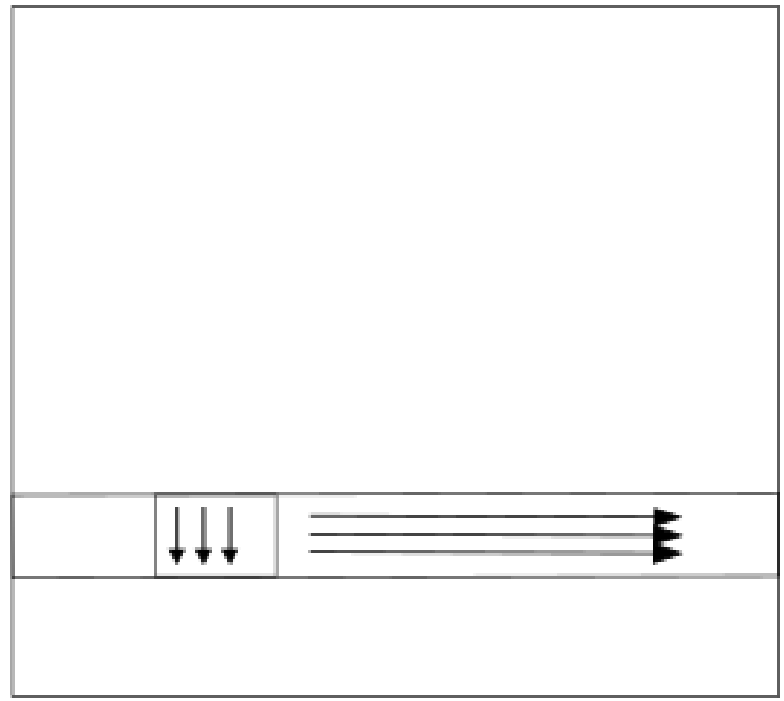
returns non-zero lower bound.

Transposition

- Basic block of FFT
- Before: Each process stores a block column,
- After: Each process stores a block row

in a picture





Exercise 5 `transposeblock`

Fill in the missing bits of the skeleton code.